BACKEND DEVELOPER PORTFOLIO

오상민

문제를 먼저 정의하고, 대안을 비교해 선택 근거를 남깁니다.

적용한 뒤에는 측정으로 확인하고 남은 과제를 기록합니다.

dnclsehd123@naver.com 010-3178-5463 github.com/5s-mini

INDEX

02 소개

03 **실시간 예약 시스템** / 동시성 제어

08 **스포츠 팀 매칭 플랫폼** / 인증, 캐싱, 동기화

13 **모의주식 투자 플랫폼 ZeroRisk** / 외부 연동, 배치

21 업무 강점

2026

단일 서버에서는 보이지 않는 문제를 먼저 짚어보는 백엔드 개발자 오상민입니다.



2001.10.15

dnclsehd123@naver.com

github.com/5s-mini

예약이 취소됐다, 목록이 안 뜬다 같은 증상에서 출발해 부하 테스트로 상황을 재현하고, 취소 사유와 설정값, 호출 경로를 따라가며 원인을 좁혀 왔습니다. 원인을 찾은 뒤에도 가장 단순한 방법부터 의심하며 선택지를 비교했고, 적용한 뒤에는 무엇이 좋아졌는지와 함께 어떤 한계가 남았는지를 같이 기록했습니다.

PROJECT

실시간 예약 시스템	분산 락 기반 동시성 제어 / 초과 예약 발생률 0%	2026.04 - 2026.05
스포츠 팀 매칭 플랫폼	인증, 검색 캐싱, 실시간 동기화 / DB 조회 1만 7천 회 → 조건당 1회	2026.06 - 2026.07
모의주식 투자 플랫폼	주문 정합성, 외부 API 연동, 배치 분리 / 19개 API + 4개 배치 구현	2026.07 - 2026.09

SKILLS

Language / Framework	Java, Spring Boot, Spring Data JPA, Spring Security, Redisson, WebSocket / STOMP
Data / Infra	MySQL, Oracle, Redis
Test	K6, JUnit

EDUCATION & CERTIFICATE

홍익대학교	소프트웨어융합학과 학사	2020.03 - 2026.08
프로그래머스	클라우드 기반 백엔드 엔지니어링 데브코스	2026.01 - 2026.07
자격 / 어학	ADsP (2026.06) / OPIc IH (2025.09)	

01 PROJECT

실시간 예약 시스템

대규모 트래픽을 가정한 식당 예약 서비스의
대기열과 동시성 제어 담당 / 5인 팀

Java / Spring Boot | Redis / Redisson | JPA 락 | K6

SUMMARY

재고가 1석 남은 좌석에 요청이 몰리면 정원을 넘는 예약이 생겼고, 정상적으로 처리되던 요청이 뒤늦게 취소되는 일도 있었습니다. 원인을 경합이 DB까지 그대로 도달하는 구조와 락 수명 관리 두 곳에서 찾아 단계적으로 해결했습니다.

ACTION

경합을 DB 앞단에서 줄이고 DB 락으로 최종 보증
락 수명과 해제 시점 재설계

RESULT

재고 1석에 100건 동시 요청 시 확정 1건 / 취소 99건, 초과 예약 0건

PERIOD

2026.04 - 2026.05

같은 좌석에 동시에 들어온 요청이 정원을 넘는 예약을 만들었습니다.

배경

재고를 확인한 뒤 차감하기까지의 사이에 다른 트랜잭션이 끼어들 수 있었습니다. 한 대에서는 synchronized로 막을 수 있지만, 서버를 여러 대로 늘리면 각 서버가 자기 프로세스 안에서만 동기화하므로 같은 좌석에 동시에 진입하는 것을 막지 못합니다.

확인 과정

k6로 재고 1석에 100건을 동시에 요청해 증상을 재현하고, 테스트 후 DB에서 확정된 예약 건수와 남은 재고를 직접 확인했습니다. 응답 코드만으로는 판단할 수 없어, 예약 테이블의 확정 건수가 재고를 넘는지와 재고가 음수가 되는지를 판정 기준으로 삼았습니다.

재고	동시 요청	개선 전
1석	100건	초과 예약 가능

검토한 대안

비관적 락 락 구간 내내 커넥션을 점유해 트래픽이 몰릴 때 커넥션 풀이 먼저 고갈될 위험

낙관적 락 경쟁이 심할수록 충돌과 재시도가 늘어 처리량이 급격히 하락

→ 두 방식 모두 경합 중인 요청이 전부 DB까지 도달한다는 점이 공통 원인

분산 락으로 경쟁을 걸러내고,
DB 락으로 정합성을 최종 보증하는 계층을 나눴습니다.

해결 과정

01 Redis **분산 락**으로 좌석 단위 진입을 직렬화해, DB까지 도달하는 동시 요청 수 자체를 줄였습니다.

02 경쟁이 앞단에서 걸러지므로 DB 락의 점유 시간이 짧아져, 풀 고갈 위험 없이 DB **락을 함께** 쓸 수 있었습니다.

03 분산 락만으로는 부족하다고 판단했습니다. 락이 만료되거나 예외로 빠지는 경로가 남아 있어, 예약 조회에 **비관적 락**을, 재고 엔티티에 **낙관적 락**을 더해 DB에서 최종 정합성을 보증하도록 했습니다.

요청 흐름

100건 동시 요청

↓

Redis 분산 락 / 좌석 단위 직렬화, 경쟁 감소

↓

DB 락 (비관적 + 낙관적) / 정합성 최종 보증

↓

확정 1건 / 취소 99건

한계 Redis 복제는 비동기이므로 장애 조치 중에는 두 요청이 같은 락을 얻을 수 있습니다. 분산 락은 경합을 줄이는 수단으로만 두고, 중복 예약 차단의 최종 책임은 DB 락에 남겼습니다.

락을 정상적으로 얻은 요청이 뒤늦게 취소되고 있었습니다.

배경

예약이 실패하는 건수가 예상보다 많았습니다. 취소된 예약의 사유 코드를 확인하니 재고 부족이 아니라 충돌로 인한 취소가 섞여 있었고, 락을 정상적으로 얻은 요청까지 취소되고 있었습니다.

원인

락 TTL이 3초로 고정되어 있었습니다. 처리 시간이 이를 넘기면 작업이 끝나기 전에 락이 풀려 다음 요청이 같은 좌석에 진입했고, 뒤늦게 커밋된 쪽에서 충돌이 발생했습니다. 락 해제가 트랜잭션 커밋보다 먼저 일어나는 구조인 것도 함께 확인했습니다.

검토한 대안

TTL 상향

가장 간단하지만 얼마로 올릴지가 추정에 의존하고, 장애 시 락이 오래 남아 복구가 늦어짐

메서드 종료 시 해제

해제가 커밋보다 먼저 일어나는 구간이 그대로 남음

해결 과정

락 수명

작업이 진행되는 동안 TTL을 자동 연장(기본 30초, 10초마다 갱신). 처리 시간을 미리 추정할 필요가 없고, 스레드가 죽으면 갱신이 멈춰 락이 자동 회수됨

해제 시점

트랜잭션이 완료된 뒤로 해제를 옮겨 커밋과 해제의 순서를 고정. 롤백된 경우에도 반드시 해제되도록 처리

대기 방식

락을 얻지 못한 요청을 즉시 취소하지 않고, 해제 알림을 받을 때까지 대기하도록 변경

100명이 동시에 요청해도 확정은 1건,
조기 해제로 인한 불필요한 취소는 사라졌습니다.

K6 동시성 테스트

	BEFORE	AFTER
초과 예약	가능	**0건**
확정 / 취소 (재고 1, 요청 100)	보장 안 됨	**1 / 99**
조기 해제 충돌 구간	존재	**제거**
락 획득 실패 시	즉시 취소	**Pub/Sub 대기**

결과

01 분산 락은 경합을 줄이는 역할, DB 락은 정합성의 최종 보증 역할로 계층을 나눠 성능과 안정성을 함께 확보

02 경합만으로 취소되던 요청이 대기 후 정상 처리되며 불필요한 예약 취소 감소

남은 과제 대기로 전환한 만큼 경합 구간에서 점유되는 스레드가 늘어납니다. 현재 규모에서는 문제되지 않았으나, 대기 상한과 스레드 풀 크기를 함께 조정해야 할 지점으로 보고 있습니다.

02 PROJECT

스포츠 팀 매칭 플랫폼

인증, 시설 검색, 실시간 동기화 담당 / 4인 팀 (팀장)

JWT / RTR | Redis Caching | WebSocket / STOMP | K6

SUMMARY

서버를 늘렸을 때 드러나는 문제 세 가지를 다뤘습니다. 로그아웃한 토큰이 다른 서버에서 통과하고, 같은 검색이 매번 DB로 향하고, 한 서버의 이벤트가 다른 서버 사용자에게 닿지 않는 상황이었습니다.

ACTION

RTR 기반 인증 및 토큰 무효화 설계
동일 조건 검색의 Redis 캐싱
STOMP + Redis Pub/Sub 브로드캐스트

RESULT

부하 테스트에서 DB 조회 약 1만 7천 회 → 조건당 1회

LEADING

데일리 스크럼 운영, PR / 이슈 템플릿 컨벤션 도입

PERIOD

2026.06 - 2026.07

토큰이 탈취되더라도 피해가 제한되도록 저장 위치와 수명을 나눴습니다.

배경

로그인 상태를 유지하려면 토큰을 클라이언트에 두어야 하는데, 스크립트가 읽을 수 있는 곳에 저장하면 탈취 시 계정이 그대로 넘어갑니다. 또 서버를 여러 대로 늘리면 각 서버가 토큰 상태를 따로 들고 있어, 로그아웃한 토큰이 다른 서버에서는 계속 통과합니다.

해결 과정

저장 위치 분리	RefreshToken은 스크립트가 읽을 수 없는 HttpOnly Cookie로 발급
수명 최소화	AccessToken은 30분 만료로 노출 시간을 줄임
재사용 감지	저장된 것과 다른 RefreshToken이 제시되면 탈취로 간주해 강제 로그아웃
상태 공유	토큰 상태를 서버 메모리가 아닌 Redis에 두어 서버가 여러 대여도 동일한 판단을 내리도록 구성
로그아웃 처리	로그아웃 및 탈퇴 시 해당 AccessToken을 남은 만료 시간만큼 Redis에 무효 목록으로 등록하고, 인증 필터가 매 요청 이를 확인해 어느 서버로 요청이 가더라도 즉시 차단

한계 AccessToken 만료 시간을 줄일수록 안전하지만 재발급 요청이 늘어납니다. 30분은 두 요소를 절충한 값이고, 실제 사용 패턴을 측정해 다시 정할 여지가 있습니다.

같은 조건의 검색이 매번 DB로 향해 커넥션 풀을 압박하고 있었습니다.

배경

시설 검색은 지역과 종목 조합이 반복되는데도 요청마다 DB를 조회하고 있었습니다.

확인 과정

인기 조건 다섯 개에 요청의 95%를 몰아 k6로 부하를 걸고, 같은 조건에서 발생한 DB 조회 횟수를 셌습니다. 조회 횟수가 요청 수와 거의 같았고, 조회가 커넥션을 점유하는 시간만큼 다른 도메인의 요청이 커넥션을 기다리게 되는 구조였습니다.

부하 테스트 중 DB 조회

약 17,000회

인기 조건 5개에 집중된 요청

95%

검토한 대안

로컬 캐시	도입은 간단하지만 인스턴스마다 값이 달라져 서버 증설 시 사용자에게 서로 다른 검색 결과가 보일 수 있음
인덱스 보강	쿼리 시간은 줄어도 커넥션 점유 횟수 자체는 그대로

→ 인스턴스 간 결과 일관성과 커넥션 점유 횟수 감소를 동시에 만족하는 방식이 필요

Redis 캐시로 약 1만 7천 회의 DB 조회를 조건당 1회로 줄였습니다.

해결 과정

01 인스턴스가 늘어도 같은 결과를 보장해야 하므로 로컬 캐시 대신 Redis **캐시**를 선택

02 TTL **5분**으로 데이터가 최신이 아닐 수 있는 구간을 제한

03 빈 결과를 캐시하면 시설이 없던 조합이 TTL 동안 빈 목록으로 유지되어 신규 등록이 반영되지 않으므로, **빈 결과는 캐시에서 제외**

결과

BEFORE / DB 조회

약 17,000회

AFTER / DB 조회

조건당 1회

한계

캐시를 Redis에 두어 인스턴스가 늘어도 모든 사용자가 같은 결과를 받습니다. 다만 결과가 있던 조합은 TTL이 지나기 전까지 신규 시설이 반영되지 않으므로, 최대 5분의 지연을 감수하는 선택이었습니다.

접속한 서버가 달라도 같은 이벤트를 받도록 전달 경로를 분리했습니다.

배경

WebSocket 연결은 접속한 서버 한 대에만 묶입니다. 서버를 여러 대로 늘리면 A 서버에 접속한 사용자가 매치에 참가해도 B 서버에 접속한 사용자 화면에는 반영되지 않습니다. 같은 매치를 보고 있는데 인원 수가 서로 다르게 보이는 상황입니다.

검토한 대안

서버 간 직접 호출	서버가 늘어날 때마다 연결 대상이 늘어 관리가 복잡해짐
단일 서버 고정	세션을 한 서버에 몰면 문제는 사라지지만 증설이 불가능

해결 과정

01 매치 참가 및 퇴장 이벤트를 **매치 단위** Redis **채널**로 발행

02 모든 서버가 해당 채널을 구독하고, 수신한 이벤트를 자기 서버에 연결된 사용자에게 STOMP **토픽**으로 전달

03 이벤트 발행과 전달을 분리해, 서버가 몇 대든 같은 이벤트가 모든 사용자에게 도달

A 서버 / 매치 참가 이벤트 발생

↓

Redis Pub/Sub / 매치 단위 채널로 발행

↓

모든 서버 구독 / 각 서버가 자기 세션에 STOMP 전달

한계 이벤트 전달을 Redis에 의존하므로 Redis 장애 시 실시간 반영이 멈춥니다. 화면 진입 시 현재 상태를 다시 조회하도록 해 복구 후 정합성은 맞도록 했습니다.

03 PROJECT

모의주식 투자 플랫폼 ZeroRisk

외부 증권 API 연동과 주문, 포트폴리오 도메인 담당 / 3인 팀

Java / Spring Boot | KIS OpenAPI | WebSocket | Scheduler

SUMMARY

돈과 수량이 오가는 도메인이라 정합성이 가장 큰 과제였습니다. 잔고보다 많은 주문이 접수되는 문제와 이미 취소된 주문이 체결되는 경합을 해결하고, 외부 증권사 API의 지연과 배치 실패가 서비스 전체로 번지지 않도록 격리했습니다.

ACTION

대기 주문을 반영한 가용 잔고 검증
체결 시점 재검증으로 취소/체결 경합 차단
배치 실패 범위를 건 단위로 격리
외부 호출 대기 상한 및 타임아웃 설계

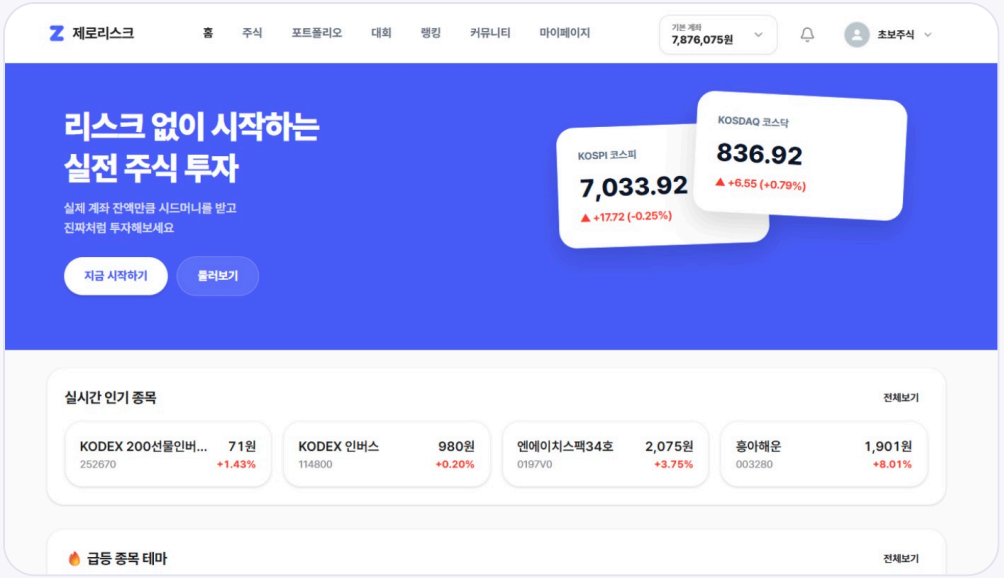
RESULT

담당 도메인에 테스트 클래스 23개 / 테스트 121개 작성, 외부 지연의 전이 경로 제거

PERIOD

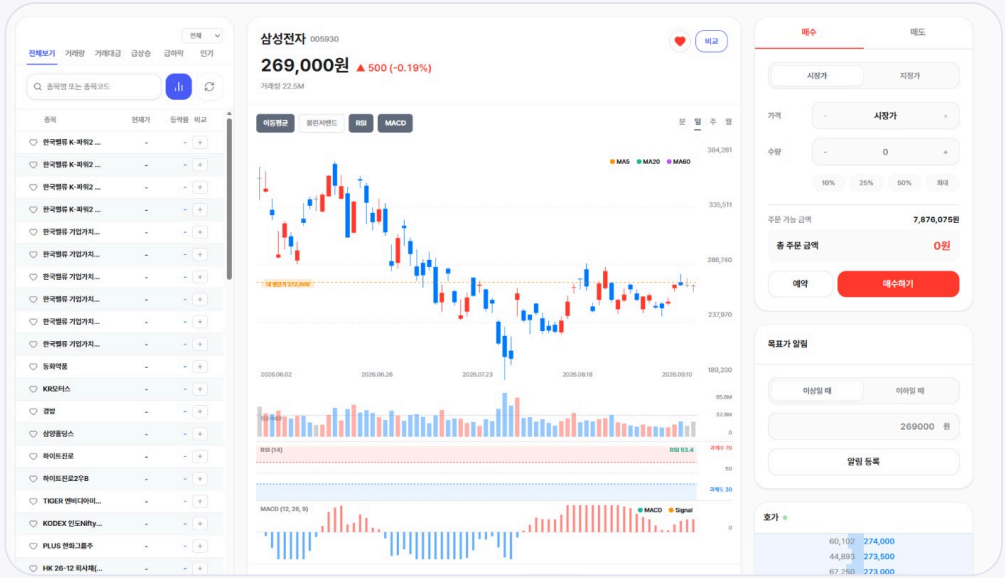
2026.07 - 2026.09

Service Screens



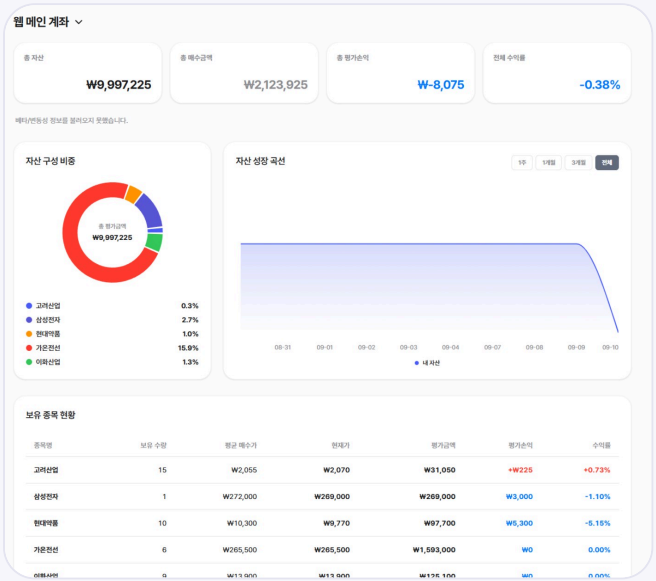
메인 페이지

KOSPI / KOSDAQ 지수, 실시간 인기 종목



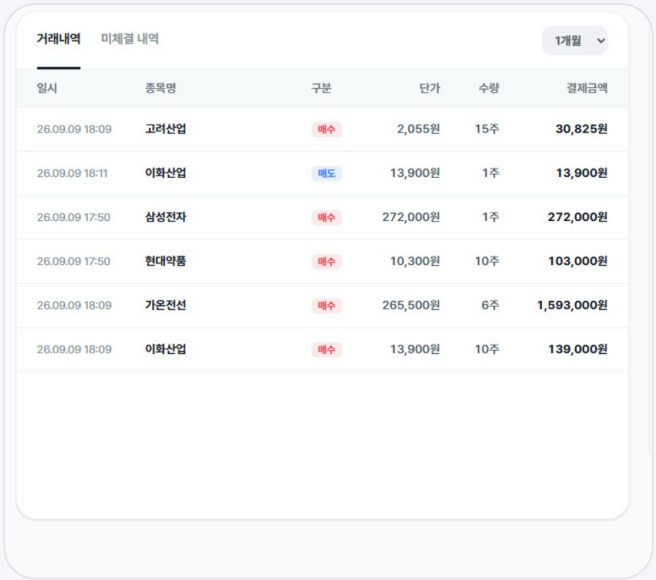
종목 상세 / 주문

캔들 차트, 보조지표, 호가, 매수 / 매도



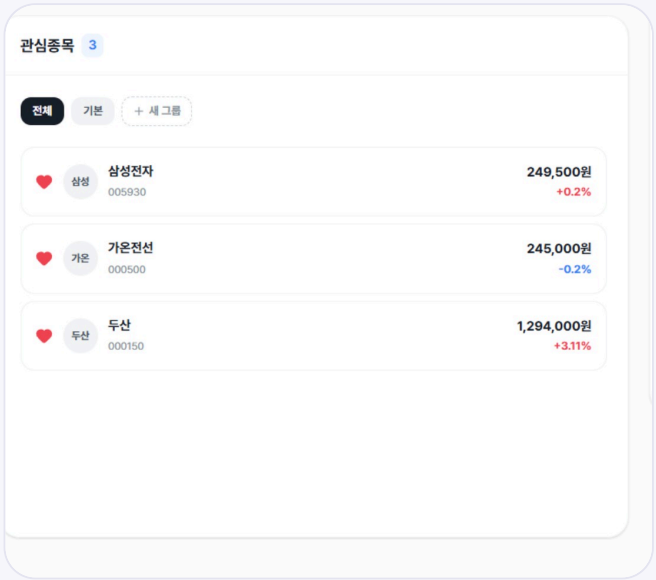
포트폴리오

자산 구성, 성장 곡선



거래내역

체결 / 미체결 내역



관심종목

그룹별 관심종목, 실시간 시세

모의주식 투자 플랫폼 / 담당 도메인 화면 (메인, 시세, 주문, 포트폴리오, 관심종목)

잔고가 1만 원인 계좌가 1만 원짜리 주문을 두 번 접수했습니다.

배경

잔고 1만 원인 계좌로 1만 원짜리 지정가 매수를 넣고, 체결되기 전에 같은 주문을 다시 넣으니 두 건 모두 정상 접수되었습니다. 첫 주문은 체결되었지만 두 번째는 영원히 체결되지 않은 채 남았습니다. 매도도 같았습니다. 10주를 보유한 계좌가 10주 매도를 여러 번 접수할 수 있었습니다.

원인

주문 접수 시점의 검증 조건과 주문 데이터를 확인했습니다. 검증은 계좌의 현재 잔고와 보유 수량만 보고 있었고, 이미 접수되어 대기 중인 지정가 주문이 쓸 예정인 금액과 수량은 반영하지 않았습니다. 잔고는 체결 시점에만 줄어드는 값이라, 체결 전까지는 같은 금액에 대한 중복 주문이 열려 있는 상태였습니다.

검토한 대안

접수 시점에 잔고 차감 + 예약 금액 별도 관리

검증 로직은 단순해지지만 체결, 취소, 미체결 종료마다 반환 처리가 필요하고, 한 경로라도 누락되면 잔고가 실제와 달라짐

잔고는 유지하고 검증 시점에 대기 주문을 합산

잔고가 변하는 지점을 체결 시점 하나로 유지할 수 있어 이 방식을 선택

해결 과정

01 매수는 대기 중인 지정가 매수 주문의 금액 합계를 잔고에서 빼고, 매도는 해당 종목의 대기 매도 수량 합계를 보유 수량에서 빼서 가용분을 계산

02 두 주문이 동시에 들어오면 합산 결과가 반영되기 전의 잔고를 각각 읽을 수 있으므로, 계좌 행에 비관적 락을 건 뒤 검증을 수행

03 주문마다 합산 조회가 추가되는 부작용이 있어, 조회 조건을 계좌와 주문 상태로 제한해 대기 주문만 읽도록 구성

한계 대기 주문이 크게 늘어나면 합산 조회 비용도 함께 커집니다. 현재 규모에서는 문제되지 않았으나, 그 시점에는 예약 금액을 별도로 집계해 두는 구조로 옮겨야 한다고 보고 있습니다.

사용자가 취소한 주문이 그대로 체결될 수 있었습니다.

배경

예약 주문 체결 배치는 대기 주문을 조회한 뒤 종목별 시세를 확인하고 체결합니다. 그런데 조회와 체결 사이에 사용자가 해당 주문을 취소할 수 있습니다. 배치가 들고 있는 것은 조회 시점의 상태이므로, 그대로 체결하면 이미 취소된 주문이 체결됩니다.

원인

접수 검증에서 겪은 문제와 같은 성격이었습니다. 읽은 시점의 값과 쓰는 시점의 값이 다를 수 있는데, 그 사이를 아무것도 막지 않고 있었습니다. 배치가 대기 주문을 락 없이 조회하고 있어, 조회 결과가 오래된 상태일 가능성을 열어 둔 구조였습니다.

검토한 대안

조회 단계부터 락

배치가 대상 전체를 오래 점유해, 그동안 사용자의 취소 요청이 대기하게 됨

체결 직전에만 재조회 후 락

점유 구간을 건별로 짧게 유지할 수 있어 이 방식을 선택

해결 과정

01 체결 직전에 주문을 락과 함께 다시 조회하고, 여전히 대기 상태인지 확인

02 계좌도 락과 함께 다시 조회해 잔고를 재확인한 뒤 체결

03 취소 처리도 같은 락 경로를 쓰도록 맞춰, 취소와 체결이 같은 주문에 동시에 들어와도 한쪽만 성립하도록 구성

한계 건별로 락을 잡으므로 체결 대상이 많아지면 배치 시간이 늘어납니다. 현재는 5분 간격 실행 안에서 처리되지만, 대상이 늘면 분할 실행을 검토해야 합니다.

한 건의 실패가
배치 전체를 되돌리고 있었습니다.

배경 / 사례 1 (종목 마스터 동기화)

장 시작 전 종목 정보를 갱신하는 배치가 실패한 뒤, 종목 조회 자체가 되지 않았습니다. 확인해 보니 특정 종목의 값이 컬럼 길이를 넘겼고, 전체를 하나의 트랜잭션으로 처리하고 있어 그 한 건 때문에 갱신분 전부가 롤백되어 있었습니다.

배경 / 사례 2 (목표가 알림 발송)

같은 성격의 문제가 알림 배치에도 있었습니다. 탈퇴한 사용자의 알림 하나에서 예외가 발생하자 배치 전체가 롤백되었고, 그 알림이 남아 있는 한 이후 실행도 계속 같은 지점에서 실패했습니다.

원인

두 배치 모두 여러 건을 하나의 트랜잭션으로 묶고 있었습니다. 건별로 독립적인 작업인데 실패 범위만 전체로 묶여 있어, 한 건의 데이터 문제가 배치 전체를 멈추고 그 상태가 다음 실행까지 이어졌습니다.

해결 과정

01 종목 마스터 동기화는 종목 한 건을 각각 독립된 트랜잭션으로 저장하도록 변경. 문제가 있는 종목만 건너뛰고 나머지는 갱신됨

02 목표가 알림은 알림 단위로 예외를 잡아 실패한 건만 기록하고 다음 건을 계속 처리하도록 변경

03 건너뛴 건은 종목 코드와 알림 ID를 남겨, 다음 실행에서 같은 지점이 반복되는지 확인할 수 있도록 함

한계 실패한 건은 건너뛰기만 할 뿐 자동으로 복구되지 않습니다. 반복 실패를 알아차리려면 로그를 봐야 하므로, 일정 횟수 이상 실패한 건을 모아 알리는 장치가 필요하다고 보고 있습니다.

시세 관련 API가 한꺼번에 응답하지 않는 현상이 있었습니다.

배경

특정 시점에 현재가, 차트, 순위 조회가 동시에 응답하지 않았습니다. 서로 다른 기능인데 함께 멈추는 점이 이상해, 공통으로 거치는 경로를 따라가 보니 모두 토큰 발급을 지나고 있었습니다.

원인

토큰 발급이 한 번에 한 요청만 통과하도록 묶여 있었고, 그 안에서 외부 응답을 기다리는 시간에 제한이 없었습니다. 외부가 지연되면 차례를 기다리는 스레드가 계속 쌓여, 발급 하나의 지연이 시세 기능 전체의 중단으로 이어지는 구조였습니다.

검토한 대안

호출부 전체 비동기 전환
대기 문제는 사라지지만 시세와 차트 전반을 다시 구현해야 해 수정 범위가 과대

외부 호출 타임아웃만 적용
호출은 끊어도 차례를 기다리는 요청의 대기 시간은 제어 불가

해결 과정

대기 상한 (3초)
시간 내 차례가 오지 않은 요청은 기다리지 않고 실패로 응답

타임아웃 / 커넥션 풀
외부 호출에 연결 3초, 응답 5초 타임아웃과 커넥션 풀을 함께 설정해 무한정 길어지는 경로 제거

결과

	BEFORE	AFTER
토큰 발급 대기	무한	**최대 3초**
외부 호출 응답 대기	무한	**최대 5초 (연결 3초)**
외부 지연 시 영향 범위	시세 기능 전체 중단 위험	**해당 요청만 실패**

검증 응답하지 않는 서버를 세워 두고 정해진 시간 안에 실패하는지, 다른 스레드가 발급 중일 때 대기 시간만큼만 기다리고 실패하는지를 테스트로 확인했습니다. 외부 의존 구간에는 타임아웃과 대기 상한을 함께 둔다는 기준을 이후 작업에 적용했습니다.

종목 목록 첫 화면이 뜨기까지 40~50초가 걸렸습니다.

배경 / 원인

목록 화면이 전체 종목의 시세를 외부 API로 하나씩 순차 조회하고 있었습니다. 종목 수만큼 외부 왕복이 그대로 누적되는 구조라, 종목이 늘수록 선형으로 느려졌습니다. 화면에 실제로 보이는 것은 스무 건 남짓인데 전부를 조회하고 있다는 점이 문제였습니다.

검토한 대안

시세 사전 저장 조회는 빨라지지만 실시간성이 떨어지고 갱신 주기를 따로 관리해야 함

병렬화만 적용 전체 시간은 줄어도 불필요한 호출이 그대로 남아 외부 호출 제한에 걸릴 위험

해결 과정 / 결과

01 목록을 스크롤에서 **페이지 단위(한 페이지 20종목)**로 바꿔 한 화면에 보이는 종목만 조회하도록 대상을 축소

02 해당 페이지의 20종목을 **5개씩 동시에** 조회

03 조회에 실패한 종목은 목록에서 제외해 일부 실패가 화면 전체를 막지 않도록 처리

BEFORE / 전체 종목 순차 조회

40~50초

AFTER / 20종목, 5개씩 병렬 조회

20~30초 / 호출량 감소

남은 과제

20~30초로는 줄었지만 산술적으로 기대한 수준에는 못 미쳤습니다. 원인을 외부 응답 자체가 아니라 내부 호출 구조에서 찾았습니다. 같은 데이터를 정렬만 달리해 쓰는 순위 조회가 화면 하나에서 네 번 반복 호출되고 있었고, 호출 제한을 넘겼을 때의 재시도가 제한을 다시 유발하고 있었습니다. 호출 통합과 재시도 조건 분리를 다음 개선 대상으로 정리했습니다.

외부 API 제한 안에서 19개 API와 4개 배치, 실시간 브로드캐스트를 함께 설계했습니다.

KIS 오픈API 연동 / 19 API + 4 배치

현재가, 일/주/월/분봉 차트, 거래량순위, 종목 통합 검색을 포함해 19개 API와 4개 배치 스케줄러를 설계/구현. 초당 2건인 호출 제한에 맞춰 배치 조회에 스로틀링을 적용해 제한 내에서 안정적으로 동작하도록 구성.

종목/시세 5 · 주문/거래 4 · 포트폴리오 3 · 관심종목 7 · 배치 스케줄러 4

실시간 시세 브로드캐스트

증권사 실시간 시세 연결을 사용자마다 맺지 않고 서버가 하나만 유지하도록 설계. 종목별 구독 사용자를 서버에서 관리하여 첫 사용자가 구독할 때만 증권사에 종목을 등록하고 마지막 사용자가 해지하면 해제. 같은 종목을 여러 사용자가 보더라도 증권사 등록은 한 번입니다.

배치 분리

예약 주문 체결, 목표가 알림 발송, 일별 자산 스냅샷 생성처럼 사용자 요청과 무관하게 주기적으로 처리할 작업을 스케줄러로 분리. 자산 추이 조회는 장 마감 후 미리 계산해 둔 스냅샷만 읽어 응답하도록 구현.

쉽게 결론 내지 않는 것을
가장 중요하게 생각합니다.

01 **읽은 값과 쓰는 값이 다를 수 있는 지점을 찾습니다.**

잔고 검증, 체결 직전 상태, 배치 조회 결과 모두 같은 성격의 문제였습니다. 한 곳을 고친 뒤 나머지 두 곳을 같은 기준으로 점검했습니다.

02 **증상에서 시작해 원인까지 좁힙니다.**

시세 API가 함께 멈추는 현상에서 공통 경로를 따라가 토큰 발급 구간을 찾았고, 예약 실패가 잦은 현상에서 취소 사유를 분류해 락 수명 문제를 찾았습니다.

03 **가장 단순한 방법부터 의심합니다.**

TTL을 올리는 것, 인덱스를 보강하는 것, 병렬도만 높이는 것을 먼저 검토하고 각각이 왜 부족한지 확인한 뒤에 다음 선택지로 넘어갔습니다.

04 **한계를 함께 기록합니다.**

캐시는 최대 5분의 지연을, 분산 락은 장애 조치 중 유실 가능성을 감수한 선택이었습니다. 무엇을 포기했는지 적어 두어야 다음 사람이 판단할 수 있다고 생각합니다.

05 **개선했다고 말하기 전에 측정합니다.**

k6로 재현한 뒤 DB의 실제 상태를 확인했고, 기대만큼 줄지 않은 구간은 원인을 찾아 남은 과제로 정리했습니다.